

Guía de estudio - AcademiSys

Sistema de gestión académica. **PHP puro con patrón MVC**, base de datos **MySQL**. Esta guía explica cómo está armado el proyecto y responde las preguntas típicas de examen.

1. ¿Qué hace el sistema?

Gestiona la vida académica de un instituto con **3 perfiles de usuario**:

- **Alumno**: ve sus cursos, notas, asistencia, correlativas, progreso, se inscribe a cursos y ve su título.
- **Docente**: ve los cursos que dicta, carga notas y asistencia, cierra cursadas.
- **Admin**: gestiona carreras, materias, correlativas, cursos, alumnos, docentes, otros admins y la auditoría.

2. Arquitectura MVC

MVC = Modelo - Vista - Controlador. Separa la lógica en tres capas:

Capa	Qué hace	Dónde está
Modelo	Habla con la base de datos (consultas SQL)	<code>models/</code>
Vista	Genera el HTML que ve el usuario	<code>views/</code>
Controlador	Recibe el pedido, usa modelos y elige la vista	<code>controllers/</code>

Flujo de una petición (¡pregunta clásica!)

```
Navegador
|  index.php?controller=alumno&action=misNotas
v
index.php      -> arranca sesión, carga clases, crea el Router
v
core/Router.php -> lee controller="alumno" y action="misNotas", valida
v
AlumnoController -> método misNotas(): pide datos al modelo
v
EvaluacionModel -> consulta SQL, devuelve las notas
v
render('alumno/mis_notas') -> arma header + vista + footer y manda el HTML
```

Archivos clave del núcleo (**core/** y raíz)

- **index.php** -> *Front Controller*. Punto de entrada **único**: toda URL pasa por acá. Inicia la sesión, define `BASE_PATH`, carga config y clases base, autocarga los modelos con `glob()` y le pasa el control al **Router**.

- **core/Router.php** -> lee **controller** y **action** de la URL, los **sanitiza** (solo letras), verifica que el controlador esté en una **lista blanca** (**auth, alumno, docente, admin**) e invoca el método. Si algo no existe -> **404**.
- **core/Controller.php** -> clase base de todos los controladores. Tiene:
- **render(\$vista, \$datos)**: incluye header + vista + footer (con **extract()** pasa las variables a la vista).
- **redirect(\$url)**: redirige y corta la ejecución.
- **core/Model.php** -> clase base de todos los modelos. En su constructor obtiene la **conexión PDO**.
- **config/database.php** -> clase **Database**, maneja la conexión.
- **config/auth.php** -> funciones de login y permisos.
- **core/Auditoria.php** -> helper para registrar acciones en la auditoría.

3. Conexión a la base - Patrón Singleton

config/database.php usa el **patrón Singleton**: garantiza **una sola** conexión PDO en toda la app.

```
private static ?PDO $instance = null;

public static function getConnection(): PDO {
    if (self::$instance === null) { // si no existe, la crea
        self::$instance = new PDO(...);
    }
    return self::$instance; // siempre devuelve la misma
}
```

- El constructor es **privado** -> nadie puede hacer **new Database()** desde afuera.
- Opciones de PDO importantes:
- **ERRMODE_EXCEPTION** -> los errores SQL lanzan excepciones (se pueden **try/catch**).
- **EMULATE_PREPARES = false** -> usa *prepared statements reales* del motor.

Por qué Singleton: abrir una conexión por cada consulta sería costoso. Con Singleton se reutiliza la misma durante toda la petición.

4. Seguridad: login, sesión y permisos

Login (**AuthController::login** + **UsuarioModel::login**)

1. El usuario manda email y contraseña.
2. **UsuarioModel::login()** busca el usuario por email y compara con **password_verify(\$pass, \$hash)**.
3. Las contraseñas se guardan **hasheadas con bcrypt** (**password_hash**), **nunca en texto plano**.
4. Si es correcto, se guardan en **\$_SESSION** los datos del usuario y la **lista de permisos** de su rol.

Roles y permisos (RBAC)

El control de acceso se basa en **roles y permisos** (tablas **rol, permiso, rol_permiso**):

- 3 roles: `alumno` (1), `docente` (2), `admin` (3).
- Cada permiso es un código, ej. `ver_notas_propias`, `gestionar_materias`, `cargar_asistencia`.
- `rol_permiso` une roles con permisos (relación **muchos a muchos**).

Cada acción del controlador empieza con un control:

```
requierePermiso('ver_notas_propias'); // si el usuario no tiene el permiso -> 403
```

Funciones en `config/auth.php`: - `requiereLogin()` -> exige sesión activa; también controla el **timeout de 30 min** de inactividad. - `requierePermiso($codigo)` -> exige login **y** que el rol tenga ese permiso. - `esAlumno()` / `esDocente()` / `esAdmin()` -> para mostrar/ocultar cosas en las vistas.

Inyección SQL - cómo se previene

Todas las consultas usan *prepared statements* con parámetros:

```
$stmt = $this->db->prepare("SELECT * FROM Alumno WHERE DNI = :dni");
$stmt->execute([':dni' => $dni]);
```

Nunca se concatena la entrada del usuario en el SQL -> no se puede inyectar código.

5. Piezas de base de datos para "lucirse"

Estas son las que más valoran en BBDD II.

a) Procedimiento almacenado `RegistrarNota`

Guarda una nota desde la lógica de la **base de datos**. Se llama así desde `EvaluacionModel::guardarEvaluacion`:

```
$this->db->prepare("CALL RegistrarNota(:dni, :idcurso, :tipo, :nota, :fecha, :inst)");
```

Qué hace el procedimiento (en SQL): 1. **Valida** que la nota esté entre 0 y 10; si no, lanza error con `SIGNAL SQLSTATE '45000'`. 2. Busca la **inscripción activa** del alumno en ese curso. 3. Si no existe, lanza error. 4. Si todo está bien, hace el `INSERT` en `Evaluacion`.

Ventaja: la validación vive en la base, así cualquier sistema que la use respeta la regla.

b) Triggers de auditoría (`auditinscripcion`)

Dos **triggers** sobre la tabla `inscripcion` registran automáticamente los cambios:

- `trg_AuditInscripcion_Insert` -> **AFTER INSERT**: guarda cada inscripción nueva.
- `trg_AuditInscripcion_Update` -> **AFTER UPDATE**: guarda el cambio **solo si el Estado cambió** (`IF OLD.Estado <> NEW.Estado`).

Guardan: inscripción, alumno, curso, estado anterior, estado nuevo, evento y fecha.

Punto clave: esto es auditoría **a nivel base de datos** - funciona aunque el cambio venga de fuera de la aplicación. La app no hace nada, lo hace el motor.

c) Dos niveles de auditoría

- **auditinscripcion** -> la llenan los **triggers** (nivel base de datos), solo inscripciones.
- **auditoria** -> la llena la **aplicación** (**core/Auditoria.php**), registra todo: logins, altas, bajas, modificaciones, etc. con usuario, IP y fecha.

d) Integridad referencial (Foreign Keys)

Todas las tablas son **InnoDB** con claves foráneas. Algunas con acciones en cascada: - **correlativa** y **cursohorario** -> **ON DELETE CASCADE** (si borrás la materia/curso, se borran solas). - La mayoría -> **ON UPDATE CASCADE**.

e) Transacción (en el seeder)

seed_completo.php envuelve la carga de datos en una **transacción**:

```
$db->beginTransaction();  
// ... cientos de INSERT ...  
$db->commit();           // si todo salió bien  
// si algo falla: $db->rollBack(); -> no queda la base a medias (atomicidad)
```

Concepto **ACID**: la transacción garantiza que se inserta **todo o nada**.

6. Reglas de negocio (lógica del sistema)

Año académico del alumno

Se calcula por el **año de ingreso** (**AlumnoModel::getAnioAcademico**):

```
año académico = (año actual - año de ingreso + 1) [mínimo 1, tope = duración de la carrera]
```

Ej: ingresó 2024, hoy 2026 -> 3° año.

Reglas de inscripción a un curso

Un alumno **solo** puede inscribirse a un curso si: 1. La materia es de **su carrera**. 2. La materia es de **su año o un año anterior** (nunca más adelantada). 3. **No** la tiene ya Aprobada, ni la está cursando (Activo), ni es Regular. (*Si está Libre, sí puede -> eso es recursar.*) 4. Tiene **aprobadas las correlativas** de esa materia.

Esto se filtra en **CursoModel::getCursosDisponibles** y se vuelve a validar del lado servidor en **AlumnoController::inscribirCurso** (no se confía solo en lo que muestra la vista).

Estados de una inscripción

Activo (cursando) · **Regular** (cursada aprobada, falta final) · **Aprobado** · **Libre** (desaprobada) · **Baja**.

Correlativas

Una materia puede requerir tener **aprobada** otra antes de cursarla. Siempre **dentro de la misma carrera**.

Título automático

Cuando un alumno aprueba **todas** las materias de su carrera, el sistema le emite el título solo (`TituloModel::emitirTituloSiCompleto`). Se dispara cuando el docente cierra la cursada o registra un final, y también al abrir "Mi Título" (red de seguridad).

7. Mapa de la base de datos

Agrupada por tema:

- **Estructura académica:** `carrera`, `materia`, `correlativa`, `planestudio`
- **Personas y acceso:** `alumno`, `direccion`, `docente`, `usuario`, `rol`, `permiso`, `rol_permiso`
- **Cursada:** `aula`, `curso`, `cursohorario`, `inscripcion`, `evaluacion`, `asistencia`, `tituloobtenido`
- **Auditoría:** `auditoria` (app), `auditinscripcion` (triggers)

Relaciones importantes: - Un `alumno` pertenece a una `carrera` y tiene una `direccion`. - Una `materia` pertenece a una `carrera` y tiene un `Anio` (año del plan). - Un `curso` es una materia dictada un año lectivo por un docente en un aula (con horarios). - Una `inscripcion` une un `alumno` con un `curso`; de ahí cuelgan `evaluacion` y `asistencia`. - `usuario` se vincula a `alumno` (por DNI) o a `docente` (por Legajo) - mutuamente excluyentes.

8. Preguntas típicas de examen (con respuesta)

¿Qué es MVC y por qué lo usaron?

Patrón que separa Modelo (datos), Vista (presentación) y Controlador (lógica). Ordena el código, permite cambiar la vista sin tocar la base, y facilita el mantenimiento.

¿Cómo entra una petición al sistema?

Todo pasa por `index.php` (Front Controller). Este crea el `Router`, que lee `controller` y `action` de la URL, valida y llama al método del controlador correspondiente.

¿Cómo evitan inyección SQL?

Con *prepared statements* de PDO: los datos van como parámetros (`:dni`), nunca concatenados.

¿Cómo guardan las contraseñas?

Hasheadas con `bcrypt` (`password_hash`). Al ingresar se comparan con `password_verify`. Nunca en texto plano.

¿Cómo controlan qué puede hacer cada usuario?

Con roles y permisos (tablas `rol`, `permiso`, `rol_permiso`). Cada acción llama a `requierePermiso('codigo')`; si el rol no tiene ese permiso, responde 403.

¿Para qué usan un procedimiento almacenado?

`RegistrarNota` valida la nota (0-10) y la inscripción dentro de la base antes de insertar la evaluación. La regla queda en la base, no solo en el PHP.

¿Qué hacen los triggers?

Auditan automáticamente los cambios de estado de las inscripciones (`auditinscripcion`), a nivel base de datos, sin que la aplicación tenga que hacer nada.

¿Qué es el patrón Singleton y dónde lo usan?

En `Database`: asegura una única instancia de la conexión PDO en toda la app (constructor privado + método estático `getConnection`).

¿Qué pasa si falla un INSERT en el medio de la carga de datos?

Como está en una transacción, se hace `rollback` y no queda nada a medias (atomicidad - ACID).

¿Cómo se decide a qué materias se puede inscribir un alumno?

Por carrera, por año académico (calculado desde el ingreso) y por correlativas aprobadas; no se muestran las ya aprobadas/en curso. Se valida en el modelo y otra vez en el controlador.

9. Cómo correr el proyecto

```
php -S localhost:8080 index.php
```

Abrir `http://localhost:8080` (MySQL de Laragon debe estar prendido).

Usuarios: - **Admin**: `admin@academisisys.edu.ar` / `Admin1234` - **Alumno**: email del alumno / `alumno123` - En el login hay **botones de acceso rápido** (solo en modo desarrollo, `DEV_MODE`).